

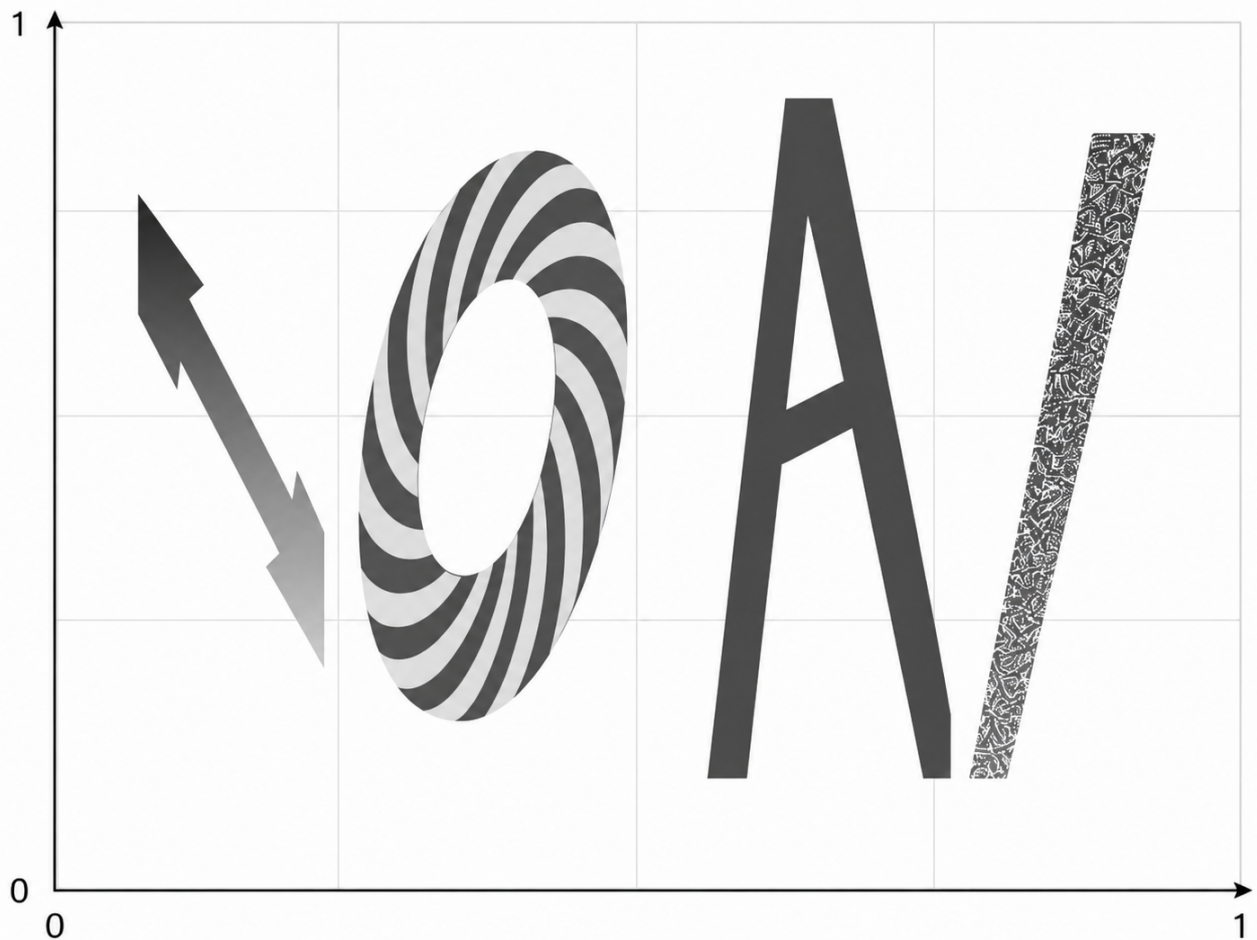
IOAI Field

- **Batas waktu:** 5 menit
- **Penyimpanan:** 5 GB
- **Ukuran solusi:** `solution.ipynb`, `custom_model.py` ≤ 1 MB secara keseluruhan
- **Model pralatih (pretrained models):** tidak ada — latih dari awal, tanpa internet saat penilaian
- **Skor Baseline:** 31.2187

Tugas

Wali Kota Astana ingin menghias kota dengan logo IOAI dengan gaya yang unik. Sebagai seorang statistikawan, ia memandang segala sesuatu—termasuk logo tersebut—sebagai fungsi spasial $F(x, y, \overline{W})$, dengan $x, y \in [0, 1]$ merepresentasikan koordinat pada bidang 2D dan $\overline{W}$ adalah himpunan parameter tersembunyi yang mendefinisikan atribut gaya seperti warna dan sudut huruf.

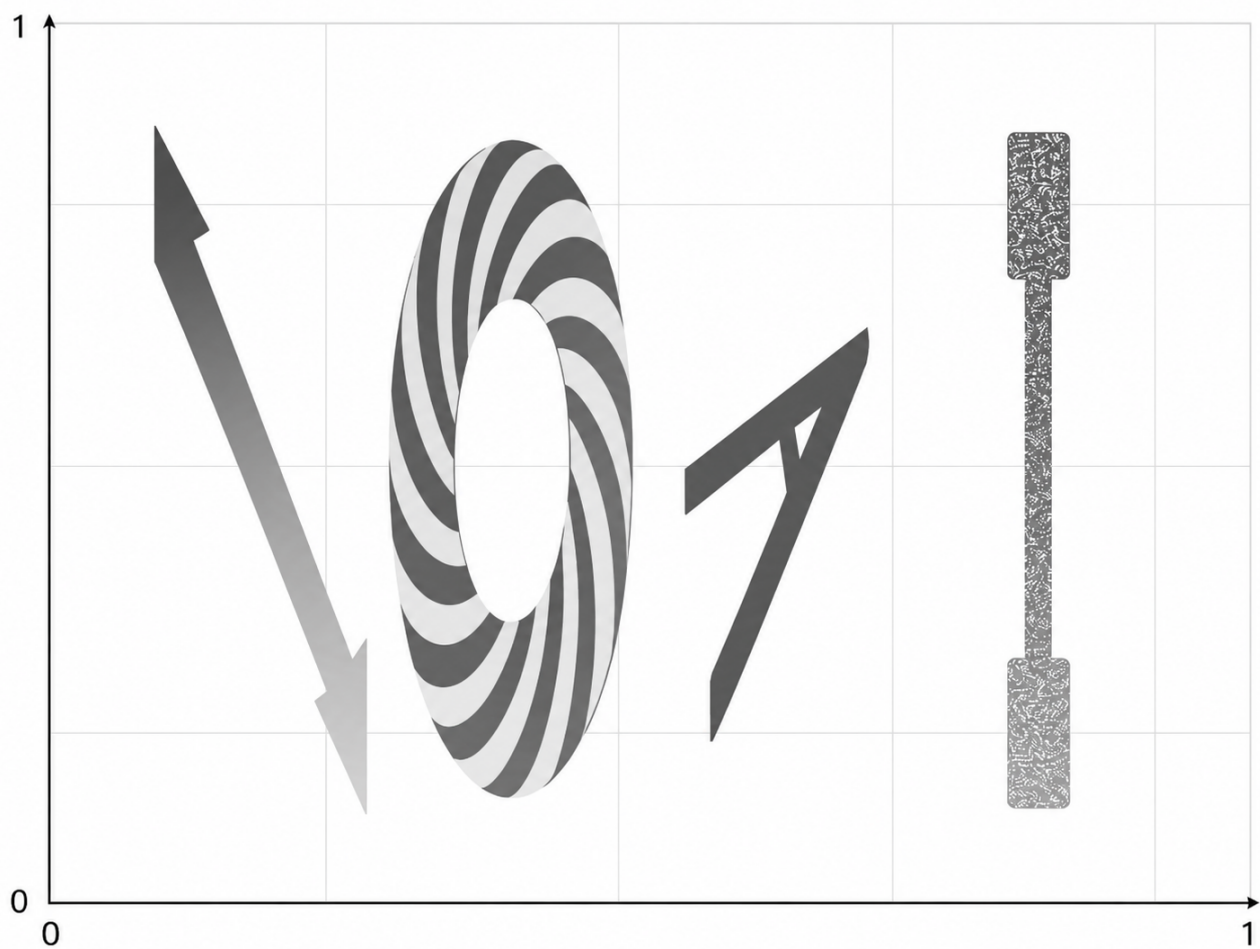
Karena F terlalu kompleks untuk dinyatakan sebagai sebuah persamaan matematika yang eksplisit, tugas Anda adalah melatih sebuah jaringan saraf tiruan (*artificial neural network*) untuk mengaproksimasi fungsi F tersebut. Jaringan tersebut akan mengeluarkan nilai **IOAI field** untuk sembarang pasangan koordinat (x, y) , menghasilkan visualisasi heatmap lengkap dari logo di seluruh bidang. Berikut adalah contoh visualisasi heatmap dari F dengan beberapa parameter tersembunyi $\overline{W}$ tertentu.

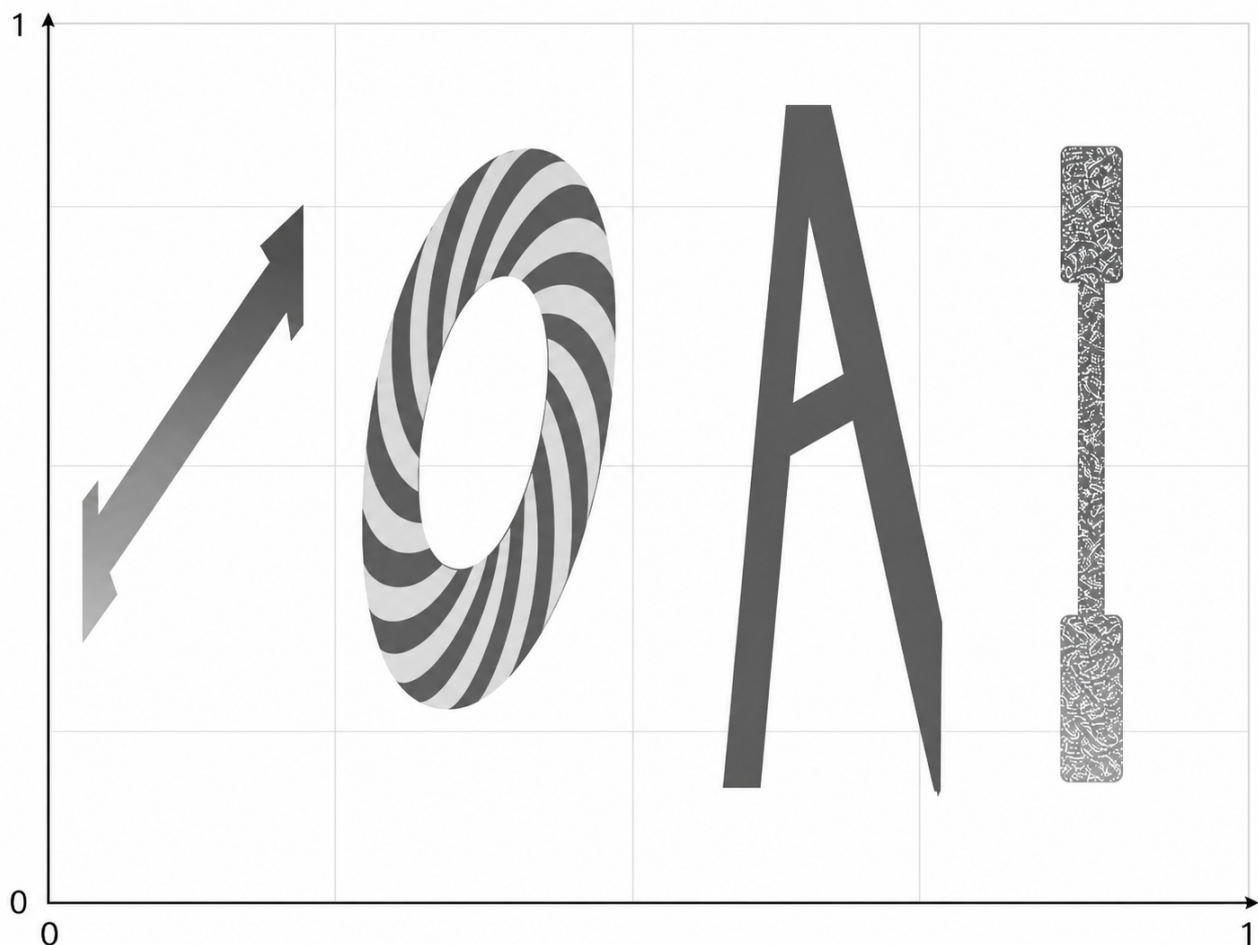


Terdiri dari apakah IOAI field itu? Empat huruf dan latar belakang.

- Nilai di dalam huruf $\mathbb{I}$ pertama adalah sangat besar ($1e+10$ dan lebih) dengan gradien linear
- Nilai pada huruf $\circ$ menunjukkan pola spiral
- Nilai di dalam huruf A selalu -1
- Nilai di dalam huruf $\mathbb{I}$ terakhir harus berupa nilai acak dengan rentang $[-2026, 2026]$, bahkan jika dievaluasi pada titik yang sama dua kali
- Di luar huruf-huruf tersebut nilainya selalu nol

Fungsi ini memiliki parameter tersembunyi $\overline{W}$, yang memengaruhi skala dan kemiringan huruf, bersama dengan rentang nilai di dalam huruf $\mathbb{I}$ pertama. Namun, huruf-huruf tersebut tidak akan berpotongan. Berikut beberapa contoh ilustratif tentang bagaimana IOAI field terlihat dengan $\overline{W}$ yang berbeda:





Apa yang diberikan kepada Anda:

Soal ini TIDAK memuat dataset. Sebagai gantinya, Anda diberikan fungsi generator yang dikonfigurasi oleh berkas konfigurasi JSON di `data/train_config/field_config.json`.

Konfigurasi uji disembunyikan, tetapi sifatnya serupa. Tugas Anda adalah melakukan *fitting* pada generator yang diberikan menggunakan sebanyak apa pun data yang Anda inginkan. Distribusi "train" dan "test" Anda dihasilkan dari generator yang sama — Anda hanya tidak tahu pada titik-titik (x_i, y_i) mana Anda akan dievaluasi.

Submission Anda harus terdiri dari:

- kelas model pelatihan yang disimpan sebagai `custom_model.py`. Model ini harus mewarisi dari kelas `torch.nn.Module` dan hanya menggunakan `import torch`. Model ini harus memuat kelas `CustomModel` yang digunakan di notebook `solution.ipynb`.
- notebook `solution.ipynb`, yang akan menghasilkan bobot `model.pt`

Penilaian

Untuk setiap region, skor minimal adalah 0 dan skor maksimal adalah 1. Skor akhir dirata-ratakan atas kelima region (empat untuk masing-masing huruf dan latar belakang) dan dikalikan dengan

100. Terdapat **penalti parameter**:

Jika model Anda memiliki lebih dari 20260 parameter, skornya dibagi dua.

Jumlah parameter diukur dengan `sum(p.numel() for p in model.parameters())`. Kami mengharapkan model yang Anda buat beroperasi dalam mode stokastik dengan menggunakan `nn.Dropout` dari PyTorch di dalam/bagian model Anda.

Untuk Region Standar

Untuk setiap region R (huruf $\mathbb{I}$ pertama, O , A , $Background$), kami mengevaluasi model pada $N_R = 512$ titik uji (x_i, y_i) dengan nilai sebenarnya v_i dan prediksi $\hat{v}_i$. Kami menggunakan Mean Absolute Error (MAE) ternormalisasi sebagai metrik utama. MAE didefinisikan sebagai:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Dan normalisasi dilakukan sebagai

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

dengan $s_R > 0$ adalah konstanta skala.

Untuk region huruf $\mathbb{I}$ terakhir

Pada region ini, **dropout diaktifkan selama evaluasi**. Untuk setiap titik uji j :

1. Kami menjalankan model sebanyak $K = 10$ kali untuk memperoleh $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Jika ada keluaran yang berada di luar rentang $[-2026, 2026]$, maka $\text{pointScore}(j) = 0$.
3. Jika tidak, hitung simpangan baku σ_j dari K keluaran tersebut dan konversikan menjadi skor:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

dengan $s_E > 0$ adalah konstanta skala tetap.

Skor region adalah rata-rata atas semua titik dalam region tersebut:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

dengan $N_E = K * N_R$.

Secara sederhana, semakin besar keragaman yang Anda miliki, semakin besar skor Anda untuk region ini. **Anda tidak boleh menggunakan random dalam bentuk murni, termasuk fungsi**

`rand*` dan `_uniform` PyTorch, keacakan harus berasal dari inferensi dengan dropout yang diaktifkan.

Cara mengirimkan

1. Buka `solution.ipynb` dan jalankan semua sel.
2. Perbaiki model `CustomModel` di `custom_model.py`
3. Pastikan sel terakhir Anda menyimpan model Anda ke berkas `model.pt`.
4. Pada tab Git di JupyterLab, lakukan stage, beri komentar, dan commit `solution.ipynb` serta `custom_model.py`, lalu push.
5. Kembali ke halaman Contest dan klik **Submit**. Komentar submit harus sama dengan komentar dari langkah sebelumnya.